

Data Engineering

Datenmodellierung, NoSQL, Data Warehouse und Lakehouse, ETL-Pipelines und Apache Spark

Dr.-Ing. Grigory Devadze

Was Sie mitnehmen

- Die Rolle des **Data Engineers** und die Bausteine einer Datenplattform einordnen
- Relationale Datenmodelle vom **ER-Modell** bis zum physischen Schema entwerfen und **normalisieren**
- **MongoDB** als Dokumentendatenbank einsetzen und gegen relationale Systeme abwägen
- **Data Warehouse, Data Lake und Lakehouse** unterscheiden und Objektspeicher wie **S3** einordnen
- **ETL-Pipelines** planen: Ingestion, Transformation, Orchestrierung und Betrieb
- **Apache Spark** mit seinen Komponenten einordnen und typische Stolperfallen kennen

Ablauf

Teil	Thema
1	Einführung in Data Engineering
2	Datenmodellierung und Normalisierung
3	NoSQL und MongoDB
4	Data Warehouse, Data Lake und Lakehouse
5	ETL-Pipelines: Grundlagen
6	Data Ingestion
7	Data Transformation
8	Orchestrierung
9	Betrieb, Stabilität und Kosten
10	Apache Spark

Teil 1: Einführung in Data Engineering

Rolle, Plattformarchitektur und Abgrenzung zu Data Science.

Was ist Data Engineering?

Data Engineering ist der Aufbau und Betrieb von Systemen, die Daten:

- erfassen,
- speichern,
- transformieren,
- verlässlich für Analyse und Produkte bereitstellen.

Kernidee:

- **Aus Rohdaten werden nutzbare Datenprodukte.**

Warum Data Engineering wichtig ist

Ohne Data Engineering entstehen häufig:

- inkonsistente Kennzahlen,
- manuelle Excel-Prozesse,
- langsame oder fehleranfällige Reports,
- geringe Vertrauenswürdigkeit von Daten.

Mit Data Engineering:

- skalierbare und reproduzierbare Datenflüsse,
- bessere Entscheidungsqualität,
- schnellere Time-to-Insight.

Rolle des Data Engineers: Überblick

Ein Data Engineer verantwortet typischerweise:

- Datenintegration aus verschiedenen Quellen,
- Entwicklung von ETL/ELT-Pipelines,
- Datenmodellierung für Analytics,
- Betrieb, Monitoring und Optimierung,
- Datenqualität und Governance-Bausteine.

Typische Aufgaben im Alltag

- Quellsysteme anbinden (API, DB, Streams)
- Ingestion und Transformation implementieren
- Datenmodelle (z. B. Fact/Dimension) entwickeln
- Ladeprozesse orchestrieren/automatisieren
- Fehleranalyse, Incident-Behebung, Performance-Tuning

Skills eines Data Engineers

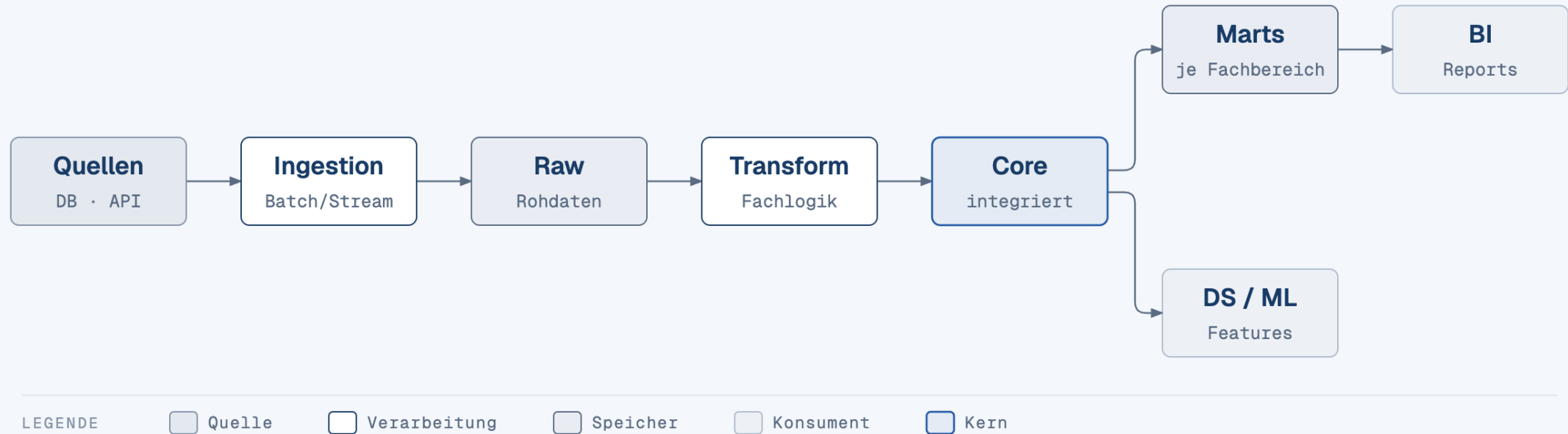
Technisch:

- SQL, Python, Datenmodellierung
- Cloud-/Storage-Konzepte, verteilte Verarbeitung
- Orchestrierung, CI/CD, Monitoring

Arbeitsweise:

- systemisches Denken,
- saubere Dokumentation,
- Zusammenarbeit mit Analytics, BI und Data Science.

Architektur von Datenplattformen



Architektur-Bausteine erklärt

- **Ingestion:** Daten zuverlässig einsammeln (batch/stream)
- **Storage:** Rohdaten + kuratierte Daten speichern
- **Transformation:** fachliche Logik, Harmonisierung, Anreicherung
- **Serving Layer:** Daten für BI, APIs, ML nutzbar machen
- **Observability:** Monitoring, Logging, Data Quality

Batch vs. Streaming in der Plattform

- **Batch:** periodische Verarbeitung, robust und planbar
- **Streaming:** ereignisnahe Verarbeitung, niedrige Latenz

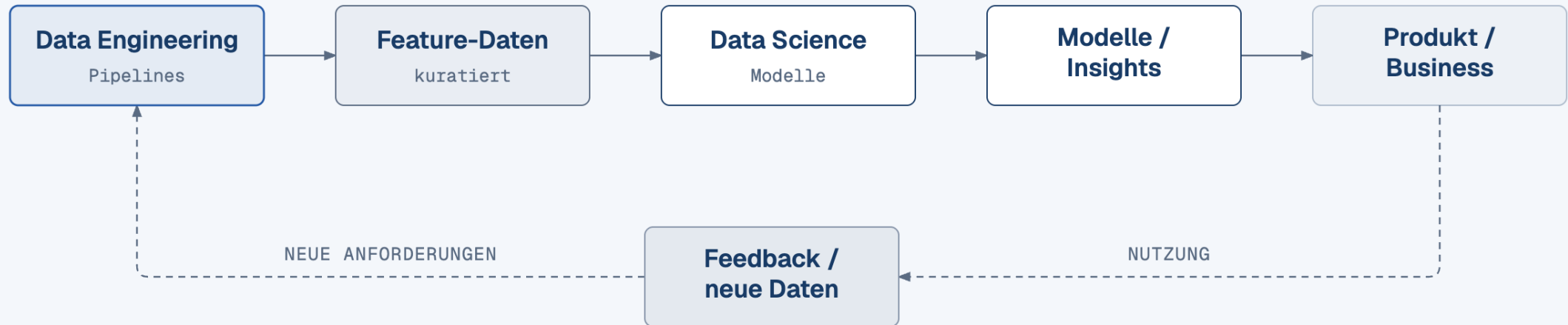
Praxis:

- viele Plattformen nutzen hybride Ansätze,
- Wahl hängt von Business-Anforderung an Datenfrische ab.

Data Engineering vs. Data Science

Aspekt	Data Engineering	Data Science
Fokus	Datenbereitstellung & Plattform	Analyse, Modelle, Erkenntnisse
Kernfrage	„Wie liefern wir verlässliche Daten?“	„Was sagen die Daten voraus/aus?“
Artefakte	Pipelines, Modelle, Datenprodukte	Features, Experimente, Modelle
Erfolgskriterium	Stabilität, Qualität, Skalierung	Modellgüte, Business-Impact

Zusammenarbeit DE + DS (Idealbild)



Häufige Missverständnisse

- „Data Engineer = nur ETL-Skripte“
- „Data Science braucht keine Plattformdisziplin“
- „Mehr Tools lösen automatisch Datenprobleme“

Real Cases aus der Praxis

- E-Commerce: Daily KPI-Reports auf nahezu Echtzeit umgestellt.
- Industrie/IoT: Sensordaten von Batch auf Streaming umgestellt.
- Finanz/Compliance: Data Lineage und Audit-Reporting stabilisiert.

Case 1: E-Commerce (Umsatz- und Funnel-Reporting)

Ausgangslage:

- KPI-Reports kamen nur 1x täglich.
- Marketing und Ops reagierten mit starker Verzögerung.

Case 1: E-Commerce (Umsatz- und Funnel-Reporting)

Data-Engineering-Maßnahmen:

- Event-Ingestion + inkrementelle Transformationen.
- Kuratierte Marts für Checkout/Funnel/Retention.
- Data-Quality-Checks auf Pflichtfelder und Duplikate.

Ergebnis:

- Reporting-Latenz von ~24h auf <30min gesenkt.
- Schnellere Kampagnenanpassungen und weniger Fehlentscheidungen.

Case 2: IoT/Produktion (Anomalie-Erkennung)

Ausgangslage:

- Maschinendaten wurden nur als Tagesbatch verarbeitet.
- Störungen wurden oft erst im Nachhinein erkannt.

Case 2: IoT/Produktion (Anomalie-Erkennung)

Data-Engineering-Maßnahmen:

- Streaming-Pipeline mit Fenster-Aggregationen.
- Trennung in Raw, Curated und Alert-Serving-Layer.
- Monitoring für Verzögerung, Ausfallrate und Datenlücken.

Ergebnis:

- Frühere Erkennung von Grenzwertverletzungen.
- Weniger ungeplante Stillstände und bessere Wartungsplanung.

Case 3: Finance/Compliance (Audit und Nachvollziehbarkeit)

Ausgangslage:

- Reports wurden aus vielen manuellen Quellen zusammengeführt.
- Hoher Abstimmungsaufwand bei Audit-Anfragen.

Case 3: Finance/Compliance (Audit und Nachvollziehbarkeit)

Data-Engineering-Maßnahmen:

- Standardisierte Transformationslogik mit versionierten Pipelines.
- Lineage von Quelle bis Kennzahl dokumentiert.
- Reproduzierbare Monats-Snapshots für Audit-Trails.

Ergebnis:

- Deutlich weniger manuelle Nacharbeit.
- Schnellere und belastbare Antworten für Revision/Compliance.

Best Practices für den Einstieg

1. Erst Datenmodell klären, dann bauen.
2. Pipeline-Tests und Datenqualitätschecks früh einbauen.
3. Ownership für Tabellen/Produkte definieren.
4. Monitoring/Latenz/Fehlerquoten transparent machen.
5. Dokumentation als Teil der Lieferung behandeln.

Zusammenfassung

- Data Engineering schafft die technische Grundlage für datengetriebene Entscheidungen.
- Data Engineers bauen und betreiben Datenplattformen als verlässliche Produkte.
- Gute Plattformarchitektur verbindet Quellen, Transformation und Konsum klar.
- Data Engineering und Data Science sind komplementär und wirken am stärksten gemeinsam.

Teil 2: Datenmodellierung und Normalisierung

Vom konzeptionellen über den logischen zum physischen Entwurf, dann die Normalformen 1NF bis 3NF.

Warum Datenmodellierung entscheidend ist

Ein gutes Datenmodell sorgt für:

- konsistente Daten,
- klare Geschäftslogik,
- wartbare Systeme,
- bessere Query-Performance.

Häufige Folgen eines schlechten Datenmodells

Ein schlechtes Datenmodell führt oft zu:

- Redundanz,
- widersprüchlichen Daten,
- teuren Workarounds in Abfragen.

Stadien der Datenbankprojektierung

- Konzeptioneller Entwurf (oder: fachliches Datenmodell)
- Logischer Entwurf
- Physischer Entwurf

Konzeptioneller Entwurf

- Aufbau eines semantischen Modells der Fachdomäne, also eines Informationsmodells auf höchster Abstraktionsebene.
- Ohne Orientierung an einer konkreten Datenbanksoftware oder einem bestimmten Datenmodell.
- Beschreibung der Informationsobjekte bzw. Fachbegriffe und der Beziehungen zwischen ihnen.
- Beschreibung von Integritätsanforderungen: zulässige Datenwerte und zulässige Beziehungen zwischen Daten.

Entity-Relationship-Modell (ER-Modell)

- Das **Entität-Beziehung-Modell** (engl. *Entity-Relationship Model*) ist ein Modell der Fachdomäne.
- Es dient dazu, die **Semantik der Fachdomäne** auf der **höchsten Abstraktionsebene** darzustellen.
- Im Fokus stehen Entitäten und die Beziehungen zwischen Entitäten.

Entität

ENTITÄT

Student

matrikelnummer

vorname

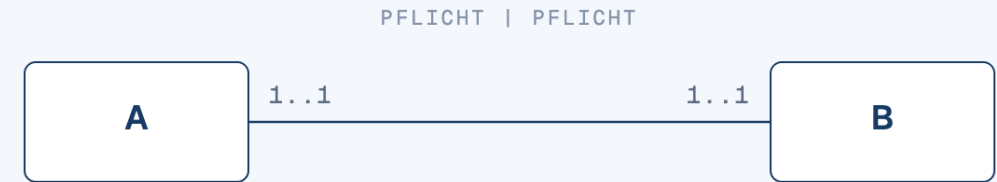
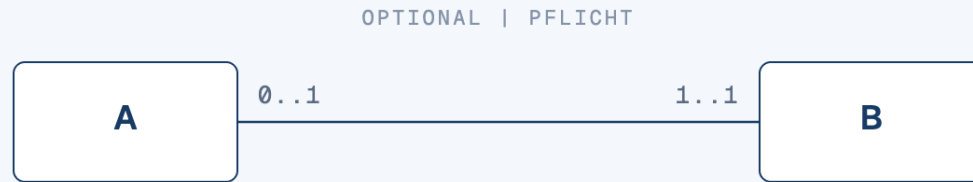
nachname

→ gruppe

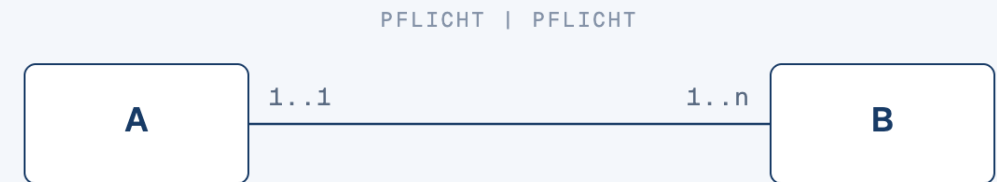
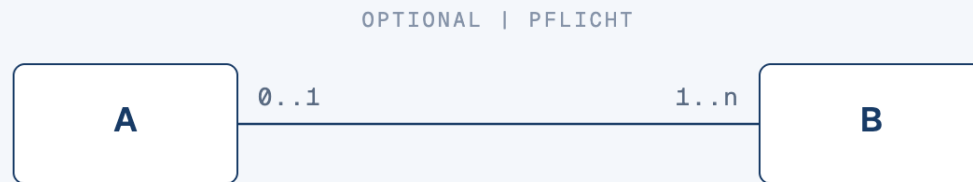
Name der Entität: ein Fachbegriff

Attribute; # = Schlüssel, → = Verweis

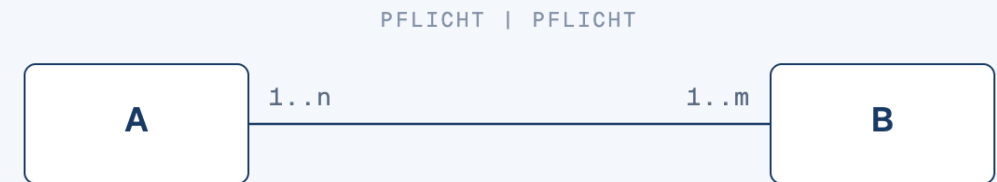
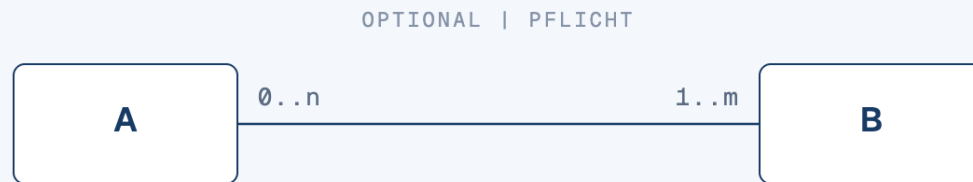
Beziehungen 1:1 (optional/pflicht)



Beziehungen 1:n (optional/pflicht)



Beziehungen n:m (optional/pflicht)



Beispiel: Fakultät, Gruppe, Student



Interpretation: Fakultät, Gruppe, Student

- **Fakultät - Gruppe:** Eine Fakultät hat $0..n$ Gruppen, eine Gruppe gehört zu $1..1$ Fakultät.
- **Gruppe - Student:** Eine Gruppe hat $0..n$ Studierende, ein Student gehört zu $1..1$ Gruppe.

Beispiel: Abteilung, Mitarbeiter, Projekt



Interpretation: Abteilung, Mitarbeiter, Projekt

- **Abteilung - Mitarbeiter:** Eine Abteilung hat $0..n$ Mitarbeiter, ein Mitarbeiter gehört zu $1..1$ Abteilung.
- **Mitarbeiter - Projekt:** Ein Mitarbeiter arbeitet an $0..n$ Projekten, ein Projekt hat $0..n$ Mitarbeiter.

Logischer Entwurf

- Erstellung eines Datenbankschemas auf Basis eines konkreten Datenmodells, z. B. des relationalen Datenmodells.
- Für das relationale Modell ist das logische Modell eine Menge von Relationsschemata, meist mit Primärschlüsseln sowie Beziehungen über Fremdschlüssel.
- In der Phase des logischen Entwurfs wird die Spezifik des gewählten Datenmodells berücksichtigt, die Spezifik eines konkreten DBMS kann jedoch noch unberücksichtigt bleiben.

Beispiel (Fakultät, Gruppe, Student)

```
FAKULTAET(  
    fakultaet_id PK,  
    name  
)  
  
GRUPPE(  
    gruppe_id PK,  
    bezeichnung,  
    fakultaet_id FK -> FAKULTAET.fakultaet_id  
)  
  
STUDENT(  
    student_id PK,  
    vorname,  
    nachname,  
    gruppe_id FK -> GRUPPE.gruppe_id  
)
```

Beispiel (Abteilung, Mitarbeiter, Projekt)

```
ABTEILUNG(  
    abteilung_id PK,  
    name  
)  
  
MITARBEITER(  
    mitarbeiter_id PK,  
    vorname,  
    nachname,  
    abteilung_id FK -> ABTEILUNG.abteilung_id  
)  
  
PROJEKT(  
    projekt_id PK,  
    titel  
)  
  
MITARBEITER_PROJEKT(  
    mitarbeiter_id FK -> MITARBEITER.mitarbeiter_id,  
    projekt_id FK -> PROJEKT.projekt_id,  
    rolle,  
    PK (mitarbeiter_id, projekt_id)  
)
```

Physischer Entwurf

- Umsetzung des Datenbankschemas für eine **konkrete** DBMS-Plattform.
- Berücksichtigt die Spezifika des gewählten Systems, zum Beispiel:
- Namenskonventionen und Namensgrenzen für Datenbankobjekte.
- Unterstützte Datentypen und deren systemabhängige Eigenschaften.

Physischer Entwurf

- Auswahl von Speicher- und Zugriffsverfahren.
- Aufteilung der Datenbank auf Dateien, Tablespaces und Geräte.
- Verteilung und Optimierung von Datenzugriffspfaden.
- Erstellung und Feinabstimmung von Indizes.
- Kurz: alles, was an das konkrete DBMS gebunden ist.

Physischer Entwurf

```
CREATE TABLE faculty (  
  id int(11) unsigned NOT NULL AUTO_INCREMENT,  
  name varchar(255) DEFAULT NULL,  
  PRIMARY KEY (id)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8;  
  
CREATE TABLE `group` (  
  id int(11) unsigned NOT NULL AUTO_INCREMENT,  
  faculty_id int(10) unsigned DEFAULT NULL,  
  name varchar(255) DEFAULT NULL,  
  PRIMARY KEY (id),  
  KEY faculty_id (faculty_id),  
  CONSTRAINT faculty_id FOREIGN KEY (faculty_id) REFERENCES faculty (id) ON DELETE SET NULL ON UPDATE SET NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8;  
  
CREATE TABLE student (  
  id int(11) unsigned NOT NULL AUTO_INCREMENT,  
  first_name varchar(255) DEFAULT NULL,  
  second_name varchar(255) DEFAULT NULL,  
  last_name varchar(255) DEFAULT NULL,  
  group_id int(10) unsigned DEFAULT NULL,  
  PRIMARY KEY (id),  
  KEY group_id (group_id),  
  CONSTRAINT group_id FOREIGN KEY (group_id) REFERENCES group (id) ON DELETE SET NULL ON UPDATE SET NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8;
```

Was bedeutet Normalisierung?

Normalisierung ist ein Entwurfsansatz für relationale Datenbanken mit dem Ziel:

- Redundanzen zu reduzieren,
- Inkonsistenzen zu vermeiden,
- Update-/Insert-/Delete-Anomalien zu verhindern,
- Datenmodelle klar und wartbar zu strukturieren.

Warum überhaupt normalisieren?

Typische Probleme ohne Normalisierung:

- **Update-Anomalie:** gleiche Information an mehreren Stellen
- **Insert-Anomalie:** neue Fakten lassen sich nicht sauber einfügen
- **Delete-Anomalie:** beim Löschen geht ungewollt Zusatzwissen verloren

Warum überhaupt normalisieren?

Nutzen:

- konsistentere Daten,
- klarere Verantwortlichkeiten,
- einfachere Pflege und Änderung.

Grundkonzepte: Schlüssel & Abhängigkeiten

Für Normalisierung zentral:

- **Schlüssel / Schlüsselkandidaten**
- **funktionale Abhängigkeit: $A \rightarrow B$**
- **voll funktional abhängig** (wichtig für 2NF)
- **transitiv abhängig** (wichtig für 3NF)

Häufiger Einwand: „Nur gesunder Menschenverstand?“

- Einwand: „Normalisierung ist nur gesunder Menschenverstand; erfahrene Profis entwerfen ohnehin intuitiv ein sauberes Schema.“
- Einordnung: Genau diese Denkregeln werden durch die Theorie der Abhängigkeiten explizit gemacht.
- Kerngedanke: Normalisierungsprinzipien sind **formalisierter** gesunder Menschenverstand.
- Nutzen der Formalisierung: Entwürfe werden nachvollziehbar, diskutierbar und systematisch prüfbar.

Normalform (1NF)

Kriterien:

- Attributwerte sind **atomar** (nicht weiter sinnvoll aufteilbar),
- keine Wiederholungsgruppen in einer Spalte oder Spaltengruppe.

Normalform (1NF)

Vor der Normalisierung (nicht 1NF):

Name	Hobbys
Thomas Müller	Schlittschuhe, Motorrad
Anna Schmidt	ClickHouse
Lukas Weber	Porträts, PlayStation
Sophie Fischer	Fotografie, Fahrräder, Audiotechnik, Messer
Daniel Wagner	Gitarre, Radiotechnik

Normalform (1NF)

Nach 1NF:

Name	Hobby
Thomas Müller	Schlittschuhe
Thomas Müller	Motorrad
Anna Schmidt	ClickHouse
Lukas Weber	Porträts
...	...
Daniel Wagner	Gitarre
Daniel Wagner	Radiotechnik

Normalform (2NF)

Kriterien:

- 1NF ist erfüllt,
- kein Nichtschlüsselattribut hängt nur von **Teilen** eines zusammengesetzten Schlüssels ab.

Interpretation:

- Nichtschlüsselattribute hängen vom **gesamten** Schlüssel ab.
- Besonders relevant bei Tabellen mit Verbundschlüsseln.

Normalform (2NF)

Vor der Normalisierung (nicht 2NF):

Name	Projekt	Projektkunde
Thomas Müller	KaiserDom	Sabine Keller
Thomas Müller	Unitrust	Jonas Winter
Anna Schmidt	Ehrenwand	Markus Adler
Lukas Weber	KaiserDom	Sabine Keller
Daniel Wagner	KaiserDom	Sabine Keller
Daniel Wagner	Unitrust	Jonas Winter

Normalform (2NF)

worker_id	Name	project_id	Projekt	Projektkunde
101	Thomas Müller	10	KaiserDom	Sabine Keller
101	Thomas Müller	11	Unitrust	Jonas Winter
...	...	...	...	...
104	Daniel Wagner	10	KaiserDom	Sabine Keller
104	Daniel Wagner	11	Unitrust	Jonas Winter

Normalform (2NF)

Nach 2NF (Zerlegung):

worker_id	project_id
101	10
101	11
102	12
103	10
104	10
104	11

Normalform (2NF)

worker_id	Name
101	Thomas Müller
102	Anna Schmidt
103	Lukas Weber
104	Daniel Wagner

project_id	Projekt	Projektkunde
10	KaiserDom	Sabine Keller
11	Unitrust	Jonas Winter
12	Ehrenwand	Markus Adler

Normalform (3NF)

Kriterien:

- 2NF ist erfüllt,
- keine transitive Abhängigkeit von Nichtschlüsselattributen.

Normalform (3NF)

Beispiel (nicht 3NF):

reifen_id	Reifenmodell	Firma	Telefon
1	Nokian Hakka	Müller Reifenhandel e.K.	+49 30 42871123
2	Nokian Nordman	Schneider Reifen GmbH	+49 40 33445566
3	Continental ContiIce	AutoMaxx GmbH	+49 89 44556677
4	Matador	Schmidt Reifenservice e.K.	+49 211 55667788

Normalform (3NF)

Problem

Transitive Abhängigkeit: `reifen_id -> Firma` und `Firma -> Telefon`, also indirekt `reifen_id -> Telefon`.

Normalform (3NF)

Nach 3NF (Zerlegung):

reifen_id	Reifenmodell	firmen_id
1	Nokian Hakka	201
2	Nokian Nordman	202
3	Continental ContiIce	203
4	Matador	204

Normalform (3NF)

firmen_id	Firma	Telefon
201	Müller Reifenhandel e.K.	+49 30 42871123
202	Schneider Reifen GmbH	+49 40 33445566
203	AutoMaxx GmbH	+49 89 44556677
204	Schmidt Reifenservice e.K.	+49 211 55667788

Grenzen & bewusste Denormalisierung

Normalisierung konkurriert teilweise mit:

- Lese-Performance,
- Einfachheit komplexer Reports,
- Domänenspezifischen Anforderungen.

Deshalb in Analytics oft:

- **gezielte Denormalisierung,**
- plus Datenqualitäts- und Abgleichmechanismen.

Zusammenfassung

- Normalisierung erhöht Datenqualität und Wartbarkeit.
- 1NF–3NF decken den Großteil praktischer Modellierungsprobleme ab.
- BCNF/4NF/5NF sind wichtig für spezielle, komplexe Abhängigkeiten.
- Gute Datenmodelle balancieren Konsistenz, Verständlichkeit und Performance.

Teil 3: NoSQL und MongoDB

Dokumentenmodell, CRUD, Aggregation und die Abwägung gegen relationale Systeme.

Was ist NoSQL?

- Sammelbegriff für Datenbanksysteme, die Daten nicht primär relational organisieren.
- Typische Ziele sind flexible Modellierung, horizontale Skalierung und verteilte Verarbeitung.
- Viele Systeme sind schemaflexibler als klassische relationale Datenbanken.
- NoSQL ersetzt SQL nicht pauschal, sondern ergänzt es für andere Problemklassen.

NoSQL-Typen

- **Dokumentenorientiert:** MongoDB, Couchbase
- **Key-Value:** Redis, DynamoDB
- **Wide-Column:** Cassandra, HBase
- **Graph:** Neo4j, JanusGraph

Praxisregel:

- Zuerst Datenzugriff und Änderungsverhalten verstehen, erst danach den Datenbanktyp auswählen.

Warum MongoDB?

- BSON-Dokumente bilden Anwendungsobjekte sehr direkt ab.
- Verschachtelte Objekte und Arrays lassen sich natürlich modellieren.
- Das Schema kann mit dem Produkt wachsen, ohne bei jeder Änderung Migrationen zu erzwingen.
- Shell, Treiber und Ökosystem machen MongoDB für Teams pragmatisch nutzbar.

Datenmodell in MongoDB

Kernbegriffe:

- **Database:** fachlicher Container
- **Collection:** Gruppe ähnlicher Dokumente
- **Document:** zusammengehörige Daten in einer Einheit
- **Field:** einzelnes Attribut im Dokument

Dokumente können:

- verschachtelte Objekte enthalten,
- Arrays enthalten,
- je nach Fall leicht unterschiedliche Felder besitzen.

Beispiel-Dokument

```
{
  "_id": "65fd1e...",
  "matrikelnummer": "S-2026-001",
  "name": {
    "vorname": "Anna",
    "nachname": "Schmidt"
  },
  "gruppe": "INF-21A",
  "tags": ["python", "tutorin"],
  "kurs_ids": ["DB-101", "AI-210"],
  "credits": 84
}
```

Embedded vs Referencing

Embedded (einbetten)

- sinnvoll, wenn Daten gemeinsam gelesen und gemeinsam gespeichert werden
- reduziert zusätzliche Abfragen
- vermeidet technische Joins in vielen Alltagsszenarien
- wird problematisch, wenn Teilobjekte stark wachsen oder unabhängig weiterverwendet werden

Referencing (referenzieren)

- sinnvoll, wenn Entitäten mehrfach genutzt werden
- reduziert Redundanz und vereinfacht zentrale Änderungen
- benötigt oft zusätzliche Abfragen oder `$lookup`
- passt gut zu Kursen, Produkten, Rollen oder Stammdaten

Referencing in der Praxis

```
// students
{
  matrikelnummer: "S-2026-001",
  name: { vorname: "Anna", nachname: "Schmidt" },
  kurs_ids: ["DB-101", "AI-210"]
}

// kurse
{
  kurs_id: "DB-101",
  titel: "Datenbanken",
  ects: 5
}
```

Viele Studierende belegen dieselben Kurse, deshalb wäre Einbetten hier meist unnötig redundant.

Arbeitskontext in der Shell

```
mongosh
use workshop
db.createCollection("students")
db.createCollection("kurse")
show collections
```

- Alle folgenden Befehle laufen in der MongoDB-Shell `mongosh`.
- `use workshop` wechselt in die Datenbank und legt sie bei Bedarf an.
- Collections entstehen explizit oder implizit beim ersten Schreiben.

CRUD in MongoDB

```
// C: Create
db.students.insertOne({
  matrikelnummer: "S-2026-002",
  name: { vorname: "Lukas", nachname: "Weber" },
  gruppe: "INF-21A",
  tags: ["java"],
  kurs_ids: ["DB-101"],
  credits: 72
});

// R: Read
db.students.find(
  { gruppe: "INF-21A" },
  { _id: 0, matrikelnummer: 1, "name.vorname": 1, "name.nachname": 1, credits: 1 }
);

// U: Update
db.students.updateOne(
  { matrikelnummer: "S-2026-002" },
  {
    $set: { credits: 78 },
    $addToSet: { tags: "mongodb" }
  }
);

// D: Delete
db.students.deleteOne({ matrikelnummer: "S-2026-002" });
```

Abfragen und Filter

// Vergleichsoperatoren

```
db.students.find({ credits: { $gte: 60, $lt: 90 } }));
```

// Kombination von Bedingungen

```
db.students.find({
  gruppe: "INF-21A",
  tags: "python"
});
```

// Sortierung, Projektion und Limit

```
db.students.find(
  { gruppe: "INF-21A" },
  { _id: 0, matrikelnummer: 1, "name.vorname": 1, credits: 1 }
).sort({ credits: -1 }).limit(5);
```

Aggregation Pipeline

```
db.students.aggregate([
  {
    $match: {
      gruppe: { $in: ["INF-21A", "INF-21B"] }
    }
  },
  {
    $group: {
      _id: "$gruppe",
      durchschnittliche_credits: { $avg: "$credits" },
      anzahl_studierende: { $sum: 1 }
    }
  },
  {
    $sort: {
      durchschnittliche_credits: -1
    }
  }
]);
```

Join-ähnliche Auswertung mit \$lookup

```
db.students.aggregate([
  { $match: { matrikelnummer: "S-2026-001" } },
  {
    $lookup: {
      from: "kurse",
      localField: "kurs_ids",
      foreignField: "kurs_id",
      as: "kurse"
    }
  },
  {
    $project: {
      _id: 0,
      matrikelnummer: 1,
      "name.vorname": 1,
      "name.nachname": 1,
      "kurse.titel": 1,
      "kurse.ects": 1
    }
  }
]);
```

Indexe in MongoDB

```
// Eindeutige Identifikation
db.students.createIndex({ matrikelnummer: 1 }, { unique: true });

// Häufige Liste: Gruppe plus Sortierung nach Credits
db.students.createIndex({ gruppe: 1, credits: -1 });

// Referenzsammlung
db.kurse.createIndex({ kurs_id: 1 }, { unique: true });
```

- Indexe beschleunigen Lesezugriffe, kosten aber Speicher und verlangsamen Schreiboperationen.
- Gute Indexe leiten sich aus echten Query-Mustern ab, nicht aus Bauchgefühl.

Schema-Validierung (optional, aber sinnvoll)

```
db.createCollection("students", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["matrikelnummer", "name", "gruppe"],
      properties: {
        matrikelnummer: { bsonType: "string" },
        name: {
          bsonType: "object",
          required: ["vorname", "nachname"],
          properties: {
            vorname: { bsonType: "string" },
            nachname: { bsonType: "string" }
          }
        },
        gruppe: { bsonType: "string" },
        tags: {
          bsonType: "array",
          items: { bsonType: "string" }
        },
        kurs_ids: {
          bsonType: "array",
          items: { bsonType: "string" }
        },
        credits: { bsonType: "int", minimum: 0 }
      }
    }
  }
});
```

Konsistenz und Transaktionen

- MongoDB unterstützt Transaktionen über mehrere Dokumente und Collections.
- Gute Modellierung versucht trotzdem, Multi-Document-Transaktionen zur Ausnahme zu machen.
- Je mehr Fachlogik mit verteilten Änderungen, desto teurer wird das System im Betrieb.

SQL vs MongoDB

- **SQL** ist stark bei stabilen, stark verknüpften Domänen mit vielen Joins.
- **MongoDB** ist stark bei dokumentnahen, hierarchischen und sich entwickelnden Datenstrukturen.
- SQL denkt stärker in Normalisierung und Beziehungen.
- MongoDB denkt stärker in Zugriffsmustern und fachlich zusammengehörigen Dokumenten.

Gute Einsatzszenarien für MongoDB

- Produkt- und Content-Kataloge mit variablen Attributen
- Event-, Telemetrie- und Log-Daten
- API-Backends mit JSON-lastigen Nutzdaten
- Anwendungen, in denen sich das Schema noch sichtbar weiterentwickelt

Wann relationale Systeme oft besser passen

- Viele komplexe Joins über stabile, stark verknüpfte Fachdomänen
- Harte Integritätsregeln über zahlreiche Entitäten hinweg
- Berichte, die stark von relationalen Auswertungen und klassischen BI-Mustern leben

Zum Schluss

- MongoDB ist besonders stark, wenn Daten fachlich natürlich als Dokumente auftreten.
- Gute MongoDB-Modellierung orientiert sich an Lese- und Schreibmustern, nicht zuerst an Normalformen.
- Flexibilität ist kein Freifahrtschein: Validierung, Indexe und Konsistenzentscheidungen bleiben Pflicht.

Entscheidungsregel: Wenn ein fachlicher Vorgang meist auf ein Dokument oder wenige eng zusammengehörige Dokumente passt, ist MongoDB oft sehr stark.

Teil 4: Data Warehouse, Data Lake und Lakehouse

OLTP und OLAP, Schichten im DWH, Inmon und Kimball, Data Lake und Lakehouse, Objektspeicher am Beispiel S3.

Warum ein Data Warehouse?

Ein DWH adressiert typische Analyseprobleme:

- Daten liegen verteilt in mehreren Quellsystemen,
- operative Systeme enthalten oft zu wenig Historie,
- analytische Fragestellungen benötigen konsolidierte, vergleichbare Daten.

Kernziel:

- **verlässliche Entscheidungsgrundlage** für Reporting, BI und Analyse.

OLTP: operative Transaktionssysteme

OLTP (Online Transaction Processing) ist auf operative Prozesse optimiert.

Typisch:

- viele gleichzeitige Nutzer,
- kurze, einfache Anfragen,
- sehr niedrige Latenz (ms bis sub-ms),
- häufige Inserts/Updates/Deletes.

OLAP: analytische Systeme

OLAP (Online Analytical Processing) ist auf Analyse optimiert.

Typisch:

- komplexe, ad-hoc Abfragen,
- historische Daten über längere Zeiträume,
- Aggregationen, Trends, Vergleichsanalysen,
- weniger Requests, aber deutlich schwerere Queries.

OLAP: analytische Systeme

Dimension	OLTP	OLAP
Ziel	Operative Prozesse	Analyse/Entscheidungen
Daten	Aktuelle Detaildaten	Historie + Aggregationen
Abfragen	Vordefiniert, kurz	Ad-hoc, komplex
Änderungen	Kontinuierlich	Periodische Ladungen / Refresh
Modellierung	Stark normalisiert	Analysefreundlich

DWH-Referenzarchitektur

1. **Primary Data Layer** (Raw/Staging)
2. **Core Data Layer** (zentraler Integrationskern)
3. **Data Mart Layer** (fachliche Ausspielung für Konsumenten)

Primary Data Layer (Raw/Staging)

Aufgaben:

- Aufnahme von Quellsystemdaten in Rohform,
- Historisierung von Änderungen,
- Entkopplung von Quelltechnologien und Folgeschichten.

Nutzen:

- Nachvollziehbarkeit, Reproduzierbarkeit, robuste Pipeline-Grundlage.

Core Data Layer

Aufgaben:

- Konsolidierung über mehrere Quellen,
- Vereinheitlichung von Schlüsseln und Strukturen,
- Datenqualitätsregeln und zentrale Transformationen.

Nutzen:

- Consumer werden von Quelllogik und Mapping-Komplexität abstrahiert.

Data Mart Layer (Konsum-Schicht)

Aufgaben:

- Aufbereitung für konkrete Analyse- und Reportingzwecke,
- themenspezifische Sichten pro Fachbereich,
- optimierte Strukturen für BI-Tools.

Nutzen:

- schnellere Time-to-Insight,
- klare, zielgruppengerechte Datenprodukte.

Inmon-Ansatz (Top-down)

Charakteristik:

- zentraler Unternehmensblick zuerst,
- starke Modellierung des Core-Layers,
- Data Marts als nachgelagerte Ableitungen.

Vorteile:

- hohe Konsistenz,
- einheitliche KPI-Definitionen,
- gute Skalierbarkeit bei vielen Quellen.

Inmon-Ansatz (Top-down)

Nachteile:

- höherer Initialaufwand,
- längere Vorlaufzeit bis zu sichtbaren Ergebnissen.

Kimball-Ansatz (Bottom-up)

Charakteristik:

- Start mit konkreten Fachanforderungen/Reports,
- Data Marts im Vordergrund,
- schrittweise Ausbau.

Vorteile:

- schneller Business-Impact,
- pragmatischer Einstieg,
- gute Iterierbarkeit.

Kimball-Ansatz (Bottom-up)

Nachteile:

- Risiko uneinheitlicher Kennzahlen,
- steigende Harmonisierungskosten bei Wachstum.

Inmon vs. Kimball

Wann eher Inmon?

- hohe Governance-Anforderungen,
- viele Quellen und Domänen,
- langfristiger Enterprise-Fokus.

Wann eher Kimball?

- schneller Nutzen gefordert,
- klar abgegrenzte Fach-Use-Cases,
- begrenzte Startkapazitäten.

Inmon vs. Kimball

Praxis:

- häufig **hybride** Umsetzung (Governance + inkrementelle Delivery).

Data Lake

Ein **Data Lake** speichert heterogene Daten oft zunächst roh:

- strukturierte Daten,
- semi-strukturierte Daten (z. B. JSON, Logs),
- unstrukturierte Daten (z. B. Bild/Video/Audio, Dokumente).

Typisch:

- schema-on-read,
- hohe Flexibilität bei neuen Datentypen,
- oft günstige, skalierbare Objekt- oder Filesystem-Storage-Basis.

Data Lake: Stärken und Grenzen

Stärken:

- schnelle Aufnahme neuer Datenquellen,
- gute Skalierbarkeit,
- niedrige Einstiegskosten.

Grenzen ohne Governance:

- unklare Semantik,
- uneinheitliche Qualität,
- hoher Aufbereitungsaufwand für BI.

Data Swamp vermeiden

Data Swamp = „Daten sind da, aber kaum nutzbar“.

- fehlende Qualitätskontrollen,
- schwache Metadatenpflege,
- keine klaren Ownerships,
- unzureichende Katalogisierung und Lineage.

Gegenmaßnahmen:

- verbindliche Ingestion-Standards,
- Data Catalog + Lineage,
- Qualitätsregeln und Zugriffskonzepte.

Warum Lakehouse?

Lakehouse entstand aus den Grenzen von reinem DWH und reinem Data Lake:

- DWH: starke Governance, aber oft teurer und schema-strenger,
- Data Lake: flexibel und günstig, aber häufig ohne ausreichende Transaktions-/Qualitätsgarantien.

Warum Lakehouse?

Lakehouse kombiniert beides:

- offene, skalierbare Storage-Schicht,
- transaktionale/verlässliche Tabellensemantik,
- einheitlicher Datenzugriff für BI, Analytics und ML.

Lakehouse: zentrale Bausteine

1. **Storage Layer** (z. B. S3/ADLS/GCS/HDFS)
2. **Open Table Formats** (z. B. Iceberg, Delta, Hudi)
3. **Catalog/Metadata Layer**
4. **Compute Layer** (Batch + Streaming)
5. **BI/Analytics Layer**
6. **Governance/Security Layer**

Lakehouse: zentrale Bausteine

Kriterium	DWH	Data Lake	Lakehouse
Datentypen	v. a. strukturiert	alle Typen	alle Typen
Governance/Konsistenz	hoch	stark variabel	hoch
Kosten/Flexibilität	mittel–hoch / mittel	niedrig / hoch	mittel / hoch
BI-Tauglichkeit	sehr gut	oft indirekt	sehr gut

Was ist S3?

- Simple Storage Service
- Objektspeicher: Daten als **Objekte** in **Buckets** (nicht „File System“)
- Zugriff über APIs/SDKs/CLI/Console (HTTP/REST)
- Hohe Haltbarkeit (Design-Ziel: 11×9) + (je nach Klasse) Multi-AZ-Resilienz (Availability Zones)
- Typische Rolle: „Separation of Storage & Compute“ (Apps, Analytics, Backup/Archive)

Datenmodell & Regionen

- **Bucket:** Container, globaler Namespace (Name muss eindeutig sein, je Partition)
- **Objekt:** Daten + Key (Name) + Metadaten; Keys sind „Pfad-artig“, aber keine echten Ordner
- **Region:** bestimmt Datenresidenz, Latenz, Kosten (u.a. Transfer/Request)
- **Naming:** DNS-kompatibel; nutze eindeutige Suffixe (z.B. GUID/Projekt/Account)

```
# Beispiel:  
export BUCKET="myproj-data-$(uuidgen | tr '[:upper:]' '[:lower:]')"  
# myproj-data-3f2c9b6d-8d74-4d4f-8f1a-6f9a2bb1c1e7
```

Versioning, Lifecycle, Replication

- **Versioning:** Schutz vor Überschreiben/Löschen (Rollback möglich)
- **Lifecycle:** automatisierte Transitions/Expiration
- **Replication:** Compliance, Latenz, Disaster Recovery; Planung inkl. Kosten/Policies

```
# Versioning aktivieren  
aws s3api put-bucket-versioning \  
    --bucket "$BUCKET" \  
    --versioning-configuration Status=Enabled
```

Use Cases

- Typische Use Cases: Static Content, Backup/Archive, Data Lake/Analytics, Event-driven Processing
- Architekturprinzip: S3 als „System of Record“; Compute/Analytics consuming on demand
- Für öffentliche Websites: häufig HTTPS und Caching davor; S3 Website-Endpunkte sind ein Spezialfall

Zusammenfassung

- OLTP und OLAP erfüllen unterschiedliche Zwecke.
- DWH lebt von klaren Schichten: Raw → Core → Mart.
- Data Lake bringt Flexibilität, braucht aber starke Governance (sonst Data Swamp).
- Lakehouse kombiniert offene Storage-Flexibilität mit DWH-ähnlicher Verlässlichkeit.
- Inmon vs. Kimball ist kontextabhängig, oft hybrid.

Teil 5: ETL-Pipelines: Grundlagen

ETL, ELT und Reverse ETL, das Grundmuster einer Pipeline und die Leitprinzipien.

Warum ETL heute geschäftskritisch ist

- Entscheidungen hängen an verlässlichen Daten
- BI, ML/AI und operative Systeme greifen auf dieselben Pipelines zu
- Compliance, Security und Auditierbarkeit steigen in Priorität

Business-Impact von guten Pipelines

- schnellere Insights
- weniger Incident-Kosten
- höhere Vertrauens Tiefe bei Stakeholdern
- kürzere Time-to-Market für Datenprodukte

ETL

- **Extract:** Daten erfassen
- **Transform:** Daten nutzbar machen
- **Load:** Daten zielgerichtet bereitstellen

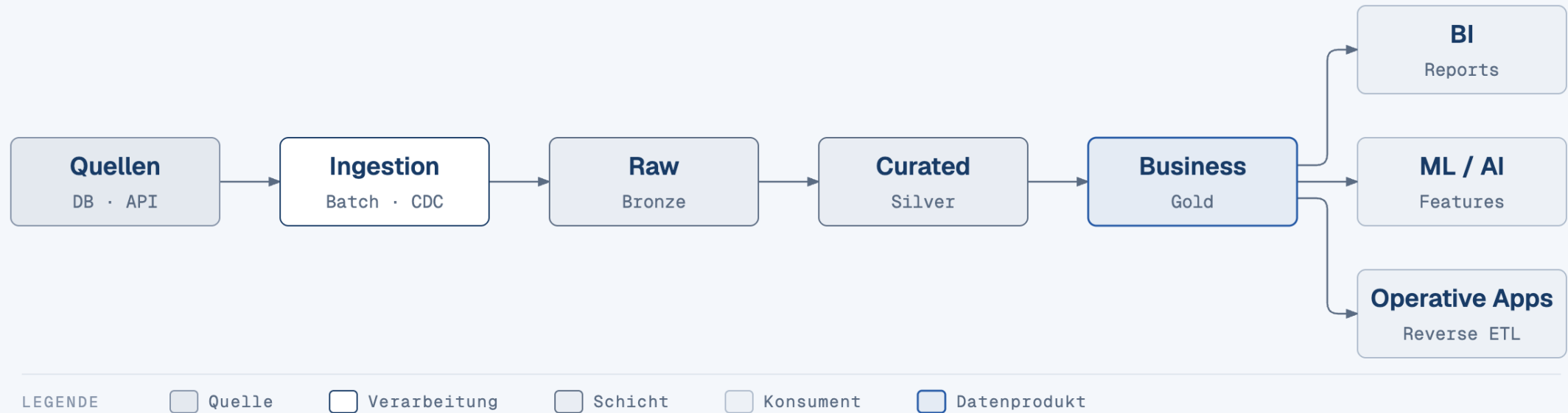
ETL ist kein Job, sondern ein System.

ETL vs ELT vs Reverse ETL

- **ETL:** Transform vor dem Laden
- **ELT:** Laden vor der Transformationslogik
- **Reverse ETL:** kuratierte Daten zurück in operative Tools

Praxis: häufig hybride Kombination.

Grundmuster



Leitprinzipien für alle Entscheidungen

- reliability first
- idempotent by default
- observability everywhere
- standardize what repeats
- optimize with metrics, not opinions

Teil 6: Data Ingestion

Ziel: viele Quellen konsistent onboarden, schema- und versionsrobust bleiben, klare Service Level Objectives (SLOs) für Frische und Vollständigkeit sichern.

Source-Assessment: Pflichtfragen

- Wer sind Owner und Stakeholder?
- Wofür wird die Quelle fachlich genutzt?
- Welche Frequenz ist real erforderlich?
- Bounded oder unbounded Datenfluss?

Source-Assessment: Pflichtfragen

- Volumen jetzt vs. in 12 Monaten?
- Format und Strukturgrad (JSON/CSV/DB/Event)?
- Qualität und bekannte Schwachstellen?
- Sicherheits-/PII-Einstufung (PII = Personally Identifiable Information)?

Ingestion-Muster

- batch (planbar, kosteneffizient)
- micro-batch
- streaming (niedrige Latenz, höherer Betriebsaufwand)
- CDC = Change Data Capture (effizient bei Änderungsdaten)

Batch vs Streaming: Entscheidungsraster

Batch wenn:

- Stunden-/Tageslatenz genügt
- Kosten und Einfachheit priorisiert sind

Streaming wenn:

- Minuten-/Sekundenreaktion nötig ist
- kontinuierliche Ereignisverarbeitung Business-kritisch ist

Ingestion-Frequenz als Produktentscheidung

- nicht „was technisch geht“
- sondern „was fachlich wirklich gebraucht wird“

Faustregel: so selten wie möglich, so häufig wie nötig.

Schema-Drift beherrschen

- Versionierung von Contracts
- Breaking Changes explizit kommunizieren
- Validierung bei Ingestion (Schema + Feldregeln)
- Quarantäne für ungültige Datensätze

Ingestion-Sicherheit

- least privilege für Jobs
- Secret Management statt Hardcoded Credentials
- Verschlüsselung in transit + at rest
- Audit-Logs für Zugriffe und Pipeline-Runs

Ingestion Anti-Patterns

- jede Quelle individuell „special cased“
- fehlende Ownership pro Source
- keine Backfill-Strategie
- Alerts erst beim Fachbereich sichtbar

Ingestion Done-Definition

- source checklist vollständig
- klare SLOs (Freshness/Completeness)
- Monitoring + Alerts aktiv
- dokumentierter Onboarding-Runbook

Teil 7: Data Transformation

Ziel: aus Rohdaten verlässliche Datenprodukte machen, Fachlogik reproduzierbar und testbar betreiben, Änderungen sicher ausrollen.

Transformation als mehrstufiger Prozess

- Rohdaten normalisieren
- Daten bereinigen und anreichern
- Fachlogik modellieren
- aggregieren und bereitstellen

Transformationsumgebung wählen

- Warehouse-zentriert (SQL-first)
- Lake + externes Compute
- Lakehouse (einheitlicherer Operating Layer)

Entscheidung nach Team, Datenprofil, Kostenmodell.

SQL vs Python

- SQL: stark für tabellarische Logik und Governance
- Python: stark für komplexe/algorithmische Verarbeitung
- starke Plattformen kombinieren beides sauber

Update-Strategien

- full refresh
- incremental insert
- upsert/merge
- CDC-basiertes Aktualisieren

Kein Pattern ist „immer richtig“.

Idempotenz

Jeder Lauf muss wiederholbar sein ohne Seiteneffekte:

- stabile Keys
- deterministische Business-Logik
- kontrollierte Merge-Policies

Ein erneuter Lauf mit denselben Inputs erzeugt denselben Zielzustand, ohne Duplikate oder widersprüchliche Updates!

Data Quality Gates in Transformation

- uniqueness checks
- null/mandatory checks
- domain checks (Wertebereiche)
- referential checks

Fehler früh stoppen!

Tests

- unit tests für Kernlogik
- data tests für Modelle
- regression checks vor Deployment
- smoke tests direkt nach Run

Transformation Anti-Patterns

- monolithische „alles in einem Job“-Pipelines
- ungeklärte Feldsemantik
- Reports auf Roh-/Silver-Tabellen
- keine Ownership pro Gold-Modell

Transformation Done-Definition

- klare Layer-Zuordnung
- dokumentierte Business-Definitionen
- automatisierte Tests/Gates
- reproduzierbarer Backfill möglich

Teil 8: Orchestrierung

Ziel: abhängige Tasks robust und sichtbar steuern, Fehlerbehandlung und Recovery standardisieren, Laufzustände nachvollziehbar halten.

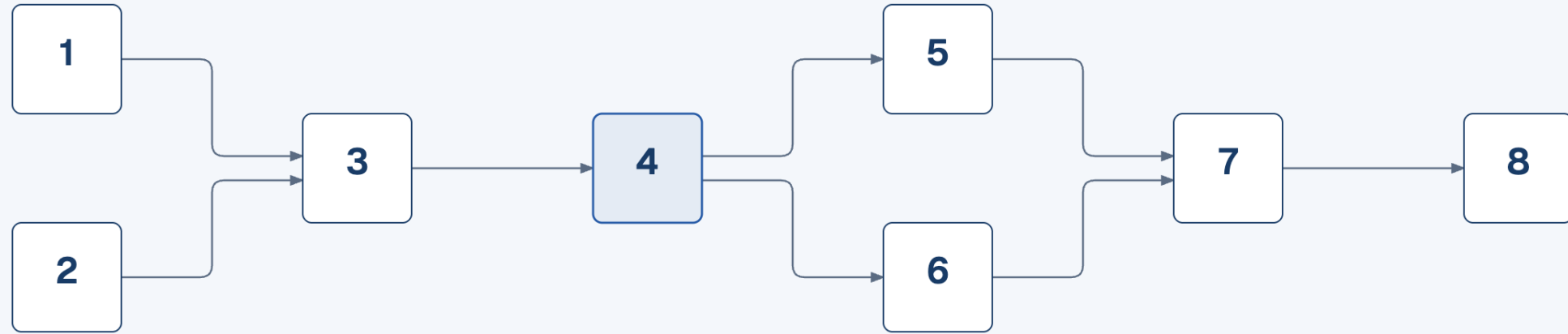
Orchestrator vs Scheduler

- Scheduler: Zeitsteuerung
- Orchestrator: Zeit + Events + Abhängigkeiten + Recovery + Monitoring

DAG-Grundlagen

- Node = Task
- Edge = Dependency
- gerichteter, zyklusfreier Ausführungsgraph

DAG-Grundlagen



KNOTEN = TASK · KANTE = ABHÄNGIGKEIT · KEINE ZYKLEN

Trigger-Modelle

- zeitbasiert (cron)
- event-driven (Webhook/Queue/File arrival)
- dependency-driven (upstream success)

Orchestrierungsmuster

- task decomposition
- retries mit backoff (wachsende Wartezeit)
- dead letter handling (separate Fehlerablage)
- idempotent reruns
- run metadata + lineage links

Rolle des Orchestrators

Best practice:

- Orchestrator steuert
- spezialisierte Services rechnen
- klare Trennung von Workflowlogik und Verarbeitungscode

Auswahlkriterien

- Skalierbarkeit
- Integrationen
- Betriebsmodell (managed/self-hosted)
- Team-Fit & Lernkurve
- Debuggability & Observability

Orchestrierung Done-Definition

- DAGs klein, modular, nachvollziehbar
- Retry-/Timeout-Konventionen etabliert
- Alerts mit Owner-Routing
- Runbook pro kritischer Pipeline

Teil 9: Betrieb, Stabilität und Kosten

Ziel: vorfälle schneller erkennen und eingrenzen, mittlere Wiederherstellungszeit reduzieren, systematisch aus Fehlern lernen.

Wartbarkeit als Wirtschaftsfaktor

- direkte Kosten: Compute, Laufzeit, Ops-Aufwand
- indirekte Kosten: Ausfallfolgen, Vertrauensverlust, Team-Burnout
- Wartbarkeit ist keine „Nice-to-have“-Eigenschaft.

Observability-Stack

- logs
- metrics
- traces
- lineage
- alerting

Quality-Metriken konkret

- uniqueness
- completeness
- validity
- consistency
- referential integrity

Lineage für Incident-Response

- schneller Root Cause
- Impact-Analyse (welche Reports betroffen?)
- kontrollierte Kommunikation an Stakeholder

Präventive Kontrollen

- anomaly detection
- data diff checks
- assertions
- schema checks bei Ingestion

Error Handling Patterns

- conditional branching
- retries mit sinnvollen Grenzen
- circuit breaker bei externen Quellen
- partial failure isolation

Graceful Degradation

Wenn ein Teil ausfällt:

- Restsystem bleibt funktionsfähig
- betroffene Produkte klar kommuniziert
- Recovery priorisiert nach Business-Impact

Incident-Lifecycle

1. detect
2. triage
3. mitigate
4. recover
5. postmortem
6. preventive action

Postmortem-Template

- was ist passiert?
- warum ist es passiert?
- warum wurde es nicht früher erkannt?
- welche Guardrails fehlen?
- welche Maßnahmen bis wann, mit Owner?

Effizienz, Skalierung, Kosten

Ziel:

- mehr Last verarbeiten
- stabile Qualität halten
- Kosten kontrolliert entwickeln

Compute-Strategien

- on-demand (stabil, teurer)
- spot/preemptible (günstiger, unterbrechbar)
- pool/shared clusters
- serverless (weniger Ops, andere Kostenkurve)

Rollen

- Platform/Data Infra
- Data Engineering
- Analytics Engineering
- Data Governance/Stewardship
- Product/Business Owner

Teil 10: Apache Spark

Spark Core, RDDs und DataFrames, Spark SQL, Streaming, MLlib und GraphX, Einordnung gegenüber Hadoop.

Was ist Apache Spark?

Apache Spark ist eine verteilte In-Memory-Plattform für Big-Data-Verarbeitung.

- Entwickelt für schnelle, parallele Datenverarbeitung in Clustern
- Unterstützt Batch, Streaming, SQL, Machine Learning und Graph-Analysen
- Open-Source-Projekt der Apache Software Foundation



Warum Spark so häufig eingesetzt wird

- **Leistung** durch In-Memory-Verarbeitung
- **Skalierbarkeit** über viele Cluster-Knoten
- **Vielseitigkeit** durch ein einheitliches Ökosystem
- **Sprachunterstützung** (u. a. Scala, Python, Java)



Spark Core als Fundament

Spark Core stellt zentrale Funktionen bereit:

- Job-Verteilung und Scheduling
- Fault-Tolerance und Wiederherstellung
- Speicherverwaltung und Ausführungsmodell
- APIs für verteilte Datenverarbeitung

Damit ist Spark Core die Basis für alle höheren Komponenten.

RDD: Das ursprüngliche Datenmodell

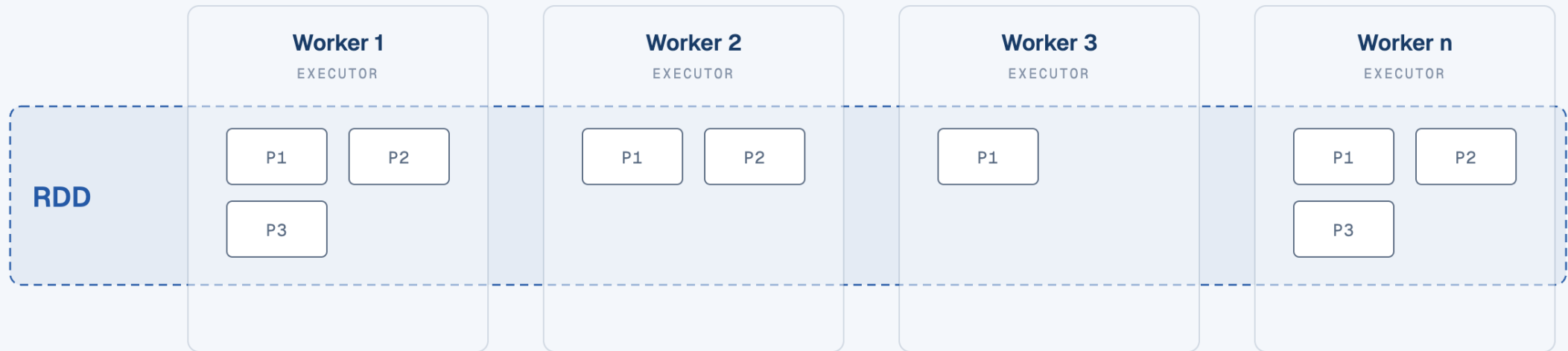
Resilient Distributed Dataset:

- Resilient: fehlertolerant
- Distributed: über Partitionen im Cluster verteilt
- Dataset: unveränderbare verteilte Datensammlung

Eigenschaften:

- Transformationen werden parallel ausgeführt
- Gut für Low-Level-Kontrolle
- Höherer Entwicklungsaufwand als bei DataFrames

RDD: Das ursprüngliche Datenmodell



PARTITIONEN (P) EINES RDD LIEGEN VERTEILT AUF DEN WORKERN

DataFrames: Produktiver arbeiten

DataFrames abstrahieren RDDs und ergänzen Schema-/Metadaten.

Vorteile:

- SQL-nahe Arbeitsweise
- Einfachere Transformationen
- Optimierungen durch Spark-Engine (Catalyst/Tungsten)
- Höhere Produktivität im Team

DataFrames: Produktiver arbeiten

RDD · KEIN SCHEMA

Verteilte Liste von Objekten,
innere Struktur für Spark unbekannt

("SE656", 6.5, "OK")

("SE6984", 6.9, "OK")

("SE764", 17.2, "OK")

("SE7231", -999.0, "ERROR")

("SE824", 11.1, "OK")

SCHEMA
→

DATAFRAME · MIT SCHEMA

Verteilte Tabelle mit benannten, typisierten
Spalten; Basis für Spark SQL

id: string	pa: double	status: string
------------	------------	----------------

SE656	6.5	OK
-------	-----	----

SE6984	6.9	OK
--------	-----	----

SE764	17.2	OK
-------	------	----

SE7231	-999.0	ERROR
--------	--------	-------

SE824	11.1	OK
-------	------	----

Spark SQL

Spark SQL ermöglicht strukturierte Analyse mit SQL und DataFrames.

- SQL-Statements direkt auf verteilten Daten
- Einheitlicher Zugriff auf CSV, JSON, Parquet, u. a.
- Integration in bestehende BI-/DWH-Landschaften via JDBC/ODBC

Spark SQL

Typischer Nutzen:

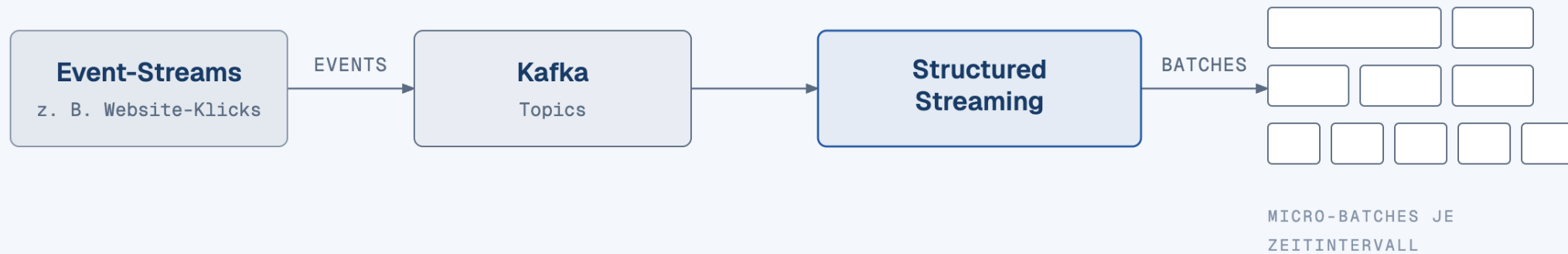
- Schnellere Entwicklung analytischer Pipelines
- Wiederverwendung vorhandener SQL-Kompetenzen im Team

Spark Structured Streaming

Structured Streaming bringt kontinuierliche Datenverarbeitung mit DataFrame-API.

- Einheitliches Programmiermodell für Batch und Streaming
- Verarbeitung von Event-Streams (z. B. Kafka)
- Geeignet für Near-Real-Time-Use-Cases

Spark Structured Streaming



MLlib: Machine Learning in Spark

Beispiele:

- Klassifikation (z. B. Random Forest)
- Regression
- Clustering (z. B. K-Means)
- Recommender (ALS)

Zusätzlich:

- Feature-Engineering
- Pipelines
- Modellpersistenz

GraphX

GraphX unterstützt Graph-Analysen direkt im Spark-Ökosystem.

- Kombination aus ETL, Analyse und Graph-Algorithmen
- Geeignet für Netzwerk- und Beziehungsdaten
- Nützlich für Fraud, Empfehlungssysteme, Knowledge Graphs

Vorteile von Apache Spark

- Sehr hohe Verarbeitungsgeschwindigkeit (insb. In-Memory)
- Breites Einsatzspektrum in einer Plattform
- Gute Integrationen zu Datenquellen und Cloud-Diensten
- Große Community und stetige Weiterentwicklung

Typische Anwendungsbereiche

- Große ETL-/ELT-Prozesse
- Datenaufbereitung für BI und Analytics
- Echtzeit-Streaming und Event-Verarbeitung
- Machine-Learning-Pipelines im produktiven Betrieb

Spark vs. Hadoop (MapReduce)

Kerngedanke:

- Spark ist oft deutlich schneller in der Datenverarbeitung
- Hadoop bringt mit HDFS ein eigenes verteiltes Dateisystem mit
- In moderner Cloud-Architektur wird Speicherung oft durch Objektspeicher ergänzt

Spark vs. Hadoop (MapReduce)

	Spark	Hadoop MapReduce
Verarbeitung	In-Memory / Festplatte	Festplatte
Geschwindigkeit	In-Memory bis zu 100-mal schneller als Hadoop	MapReduce liest und schreibt immer von der Festplatte, daher nicht so schnell
Einsetzbarkeit	Batch, Streaming, Machine Learning und Graphverarbeitung	nur Batchverarbeitung
Real-Time-Fähigkeit	ja, bis zu Millionen Einträge pro Sekunde	nein
Robustheit	Fehlertoleranz vorhanden, Anwendung muss bei Fehlern nicht zwingend neu gestartet werden	Fehlertoleranz vorhanden, Anwendung muss bei Fehlern nicht zwingend neu gestartet werden
Sicherheit	noch Verbesserungspotenzial im Vergleich zu Hadoop	bietet mehr in Bezug auf Sicherheit

Einordnung: Wo Spark stark ist

Spark ist besonders stark, wenn:

- iterative Verarbeitung nötig ist
- viele Transformationen auf großen Datenmengen laufen
- Batch + Streaming + ML kombiniert werden sollen

Hadoop/HDFS bleibt relevant, wenn ein klassisches On-Prem-Storage-Ökosystem zentral ist

Spark in der Cloud

Cloud-Nutzung reduziert oft operativen Aufwand:

- weniger Cluster-Betriebsaufwand
- schnellere Bereitstellung für Teams
- bessere Elastizität für Lastspitzen
- Databricks

PySpark

PySpark ist die Python-Schnittstelle von Spark.

Vorteile:

- Niedrige Einstiegshürde für Python-Teams
- Sehr gut für Data Science Workflows

Zu beachten:

- Je nach Workload teils langsamer als Scala-Ansätze
- Solides Verständnis für verteilte Verarbeitung bleibt essenziell

Erfolgsfaktoren

1. Datenmodell und Partitionierung sauber designen
2. DataFrame/SQL bevorzugen, RDD nur gezielt einsetzen
3. Monitoring und Kostenkontrolle von Anfang an mitdenken
4. Pipelines reproduzierbar bauen (CI/CD, Versionierung)
5. ML-Lifecycle inkl. Feature-Management planen

Grenzen und typische Stolperfallen

- Falsche Partitionierung kann Performance stark bremsen
- Zu große Shuffle-Operationen führen zu Laufzeitproblemen
- Streaming benötigt sauberes State- und Fehlerhandling
- „Spark löst alles“ ist ein Anti-Pattern: Architektur bleibt entscheidend

Zusammenfassung

Apache Spark ist eine starke, vielseitige Plattform für moderne Datenverarbeitung.

- Besonders wertvoll bei großem Datenvolumen und gemischten Workloads
- Relevante Kernbausteine: Spark SQL, Structured Streaming, MLlib, GraphX
- In Cloud-Umgebungen häufig der pragmatischste Weg zu skalierbaren Pipelines